`notify_all()`. It's therefore recommended that the overload of `wait_until()` that takes a predicate is used in preference where possible. Otherwise, it's recommended that `wait_until()` be called in a loop that tests the predicate associated with the condition variable. There's no guarantee as to how long the calling thread will be blocked, only that if the function returns `false`, then `Clock::now()` returns a time equal to or later than `absolute_time` at the point at which the thread became unblocked.

Synchronization

Calls to `notify_one(), notify_all(), wait(), wait_for()`, and `wait_until()` on a single `std::condition_variable` instance are serialized. A call to `notify_one()` or `notify_all()` will only wake threads that started waiting *prior* to that call.

Std::Condition_Variable::Wait_Until Member Function Overload That- t Takes a Predicate

Wait until the `std::condition_variable` is woken by a call to `notify_one()` or `notify_all()` and the predicate is `true`, or until the specified time has been reached.

Declaration

```
template<typename Clock,typename Duration,typename Predicate>
bool wait_until(
    std::unique_lock<std::mutex>& lock,
    std::chrono::time_point<Clock,Duration> const& absolute_time,
    Predicate pred);
```

Preconditions

The expression `pred()` shall be valid and shall return a value that is convertible to `bool`. `lock.owns_lock()` shall be `true`, and the lock shall be owned by the thread calling `wait()`.